Dev-lab webapp2 構造分析

作成日: 2026年10月 / 読了目安: 15分

現行プロジェクト アーキテクチャ徹底解説

構造設計・解決した課題・残された課題と処方箋

本プロジェクト（webapp2）は、Spring等のフレームワークを使用しない制約下で、 **「Front Controller + Command パターン + レイヤードアーキテクチャ」**を採用しています。 本稿では、現在の構造の狙い、すでにリファクタリングで解消された課題、そして今後クリーン／ヘキサゴナルへ進化させるための残課題と解決策を詳細に解説します。

目次

1. 現行アーキテクチャの全体像とデータフロー
2. 各レイヤーの責務と実装パターン
3. これまでに【解決した課題】とその方法
4. 現在【残っている課題】と具体的解決策
5. ヘキサゴナル／クリーンへの進化ロードマップ

Section 01

現行アーキテクチャの全体像とデータフロー

現在のコードベースは、古典的なJava Servlet特有の「Fat Servlet（1ファイルに画面遷移・DB・ロジックが混在する状態）」を排し、 **4つの関心事（Presentation / Application / Domain / Infrastructure）**に階層化されています。

1. プレゼンテーション層 (Presentation Layer) `com.example.controller`

HTTP受付、ルーティング、セッション/パラメータ検証、画面フォワード・リダイレクト制御

- `FrontController（Servlet）`
- `Action（ShainListAction等）`
- `View（遷移先/種別保持）`

↓ 呼び出し（ActionがServiceを実行）

2. アプリケーション層 (Application Layer) `com.example.service`

ユースケースの実現、トランザクション境界管理（コミット/ロールバック）、DAOの組み合わせ

- `ShainService / KintaiService`
- `TransactionManager`
- `TransactionalOperation<T>`

↓ 依存（ServiceがEntityを操作し、DAOに渡す）

3. ドメイン層 (Domain) com.example.entity

業務データの表現（POJO）

`Shain, Kintai, CalendarDay, LeaveGrantRule`

4. インフラ層 (Infrastructure) dao / sql / infra

DB接続、PreparedStatement、SQL実行

`ShainDao, KintaiDao, ConnectionBase, *.sql`

Section 02

各レイヤーの責務と実装パターン

① プレゼンテーション層：FrontController & Commandパターン

全てのリクエスト（*.do）を単一の FrontController が受け取ります。URL名に対応する Action（コマンドオブジェクト）を動的に特定・実行することで、コントローラーの責務を1クラス＝1ユースケースに完全に分離しています。

```
// FrontController.java: ルーティングとView制御を一元化
Action action = getAction(request);          // 例: /shainList.do -> ShainListAction
View view = action.execute(request, response); // 実行
if (view != null) {
    if (view.isRedirect()) response.sendRedirect(contextPath + view.getPath());
    else request.getRequestDispatcher(view.getPath()).forward(request, response);
} // view == null の場合は API（JSON直接出力）として扱う
```

② アプリケーション層：ラムダ式トランザクション管理

Service クラスは業務処理フローを表現します。DB更新時は TransactionManager.execute(...) を用いることで、commit()、rollback()、リソースクローズのボイラープレートをビジネスロジックから完全排除しています。

```
// ShainService.java: テンプレートメソッド＋ラムダ式による宣言的トランザクション
public void deleteShain(int id) throws SQLException, NamingException, IOException {
    transactionManager.execute(con -> {
        shainDao.deleteById(con, id); // 単一トランザクション内で実行
        return null;
    });
}
```

③ インフラストラクチャ層：外部SQLファイル分離

Javaコード内にSQL文字列を直接埋め込むのをやめ、`src/main/resources/sql/**/*.sql` へ分離。 `AbstractDao` 経由でSQLを読み込み、可読性と保守性を担保しています。

Section 03

これまでに【解決した課題】とその方法

✅ 課題1: Fat Servlet / スパゲッティコードの解消

設計パターン

【Before (過去の課題)】

単一のServlet内に「リクエスト解析」「DB接続」「SQL発行」「セッション操作」「JSPフォワード」が全て書かれ、数百行に肥大化していた。

【After (解決方法)】

Front Controller + Action (Command) パターンを採用。1つの操作につき1つのActionクラス（例: `ShainDeleteExecuteAction`）に細分化し、責務を10〜30行に限定。

✅ 課題2: トランザクション管理の不整合とコード重複

データ整合性

【Before (過去の課題)】

各DAOメソッドやServletごとに `con.commit()` や `con.rollback()` が書かれ、複数テーブル更新時に不整合が発生しやすかった。

【After (解決方法)】

`TransactionManager` を作成し、`execute(con -> { ... })` で一括管理。Service側で境界を明示し、例外時は自動ロールバックを保証。

✅ 課題3: SQLのJava内ハードコーディング

保守性・セキュリティ

【Before (過去の課題)】

Java文字列の結合でSQLが組み立てられ、構文ミスやSQLインジェクションのリスク、DBスキーマ変更時の追跡困難が発生。

【After (解決方法)】

`resources/sql/機能名/ファイル名.sql` として外部化。IDEでの構文ハイライトと再利用性を向上。

✅ 課題4: 画面遷移処理とAjax(JSON) API処理の混在

拡張性

【Before (過去の課題)】
JSPへのHTMLレンダリングと、JavaScriptからのJSON非同期リクエストの処理境界が曖昧だった。

【After (解決方法)】
通常Action（View返却）とApiAction（JSON直接送出、View=null）を明確に分割。勤怠更新（`KintaiUpdateApiAction`）などでSPA的なAjax通信を実現。

Section 04

現在【残っている課題】と具体的解決策

⚠ 残課題1: Service層がDAO具象クラスに直接依存（依存性逆転の未適用）

最重要

現状の課題: `ShainService` は `import dao.ShainDao;` を行い、具象クラスを直接参照・生成しています。 これにより、Service層の単体テストを行う際にも**実DB（PostgreSQL）またはモック設定が必須**となり、テスト実行が重く、テストコード作成のハードルが上がっています。

💡 解決策（ヘキサゴナル／DIPの導入）:

1. `ShainRepository` インターフェースをサービス側で定義（Outbound Port）。
2. `ShainDao` がそのインターフェースを実装（Driven Adapter）。
3. Serviceの単体テスト時は、メモリ内の `HashMap` を使ったダミークラスを渡すだけで、**DBなし・1ミリ秒でテストが完了**するようになります。

⚠ 残課題2: インフラ層の例外（SQLException/IOException）が上位層に露出

関心の分離

現状の課題: `ShainService` のメソッドシグネチャに `throws SQLException, NamingException, IOException` が記載されています。 DBやファイルI/Oの技術的都合（インフラの関心事）がビジネス層の公開APIに漏れ出しています。

💡 解決策:
独自の非チェック例外（例: `DomainException`, `SystemException`）を定義し、DAOやTransactionManager内でキャッチしてラップ・再スローします。 Serviceのメソッドはビジネス例外のみをスローするクリーンな状態にします。

⚠ 残課題3: 貧血ドメインモデル（Anemic Domain Model）

ドメイン駆動

現状の課題: `Shain` や `Kintai` エンティティが Getter/Setter のみの「ただのデータ入れ物」になっています。 勤務時間の計算ロジックや残業時間の妥当性判定などが、JavaScript（フロント）やService層（手続き型）に散乱しています。

💡 解決策:
「勤務時間を計算する」「年休付与条件を満たしているか判定する」といった純粋な業務ルールを、Entityクラス内のメソッド（または値オブジェクト）へ移譲・カプセル化します。

⚠ 残課題4: 旧レガシーコードの残存と二重構造

保守性

現状の課題: src/main/java/controller/ShainIndex.java や model/ShainLogic.java などの旧実装が残存しており、 新構造（com.example.*）との境界が初見の開発者にとって分かりにくくなっています。

解決策:

動作確認が取れた旧コードを安全に削除（または deprecated マークを付与）し、すべてのルーティングを com.example.controller.FrontController に一本化します。

Section 05

ヘキサゴナル／クリーンへの進化ステップ（3段階）

一気に全ての構造を書き換える必要はありません。以下の3ステップで段階的に進めることで、安全にモダンアーキテクチャへ移行できます。

ステップ 1（着手推奨）

Repository Interface の抽出

ShainService が直参照している ShainDao からインターフェース（ShainRepository）を抽出し、依存を逆転させる。

ステップ 2

例外のドメインカプセル化

Serviceメソッドから SQLException や IOException を排除し、独自のビジネス例外に統一する。

ステップ 3

旧コードの完全撤去

ルートパッケージの controller.* や model.* を削除し、com.example.* に完全統一する。